National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

RAI-833 Deep Learning

Tutorial 9: Object Detection using SSD and YOLO

Single-Stage Detector Inference and Comparison

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

April 12, 2026

1 Objectives

The primary objective of this tutorial is to explore and implement **single-stage object detection architectures**. Unlike two-stage detectors (such as Faster R-CNN) which require a separate Region Proposal Network (RPN), single-stage models predict bounding boxes and class probabilities simultaneously, making them highly suitable for real-time applications. Key learning outcomes include:

- Implementing inference using a pre-trained **SSD (Single Shot MultiBox Detector)** via PyTorch's `torchvision`.
- Implementing inference using a pre-trained **YOLO (You Only Look Once)** model via the Hugging Face `transformers` library.
- Comparing the detection confidence and localization capabilities of both architectures on a dense, real-world street scene image.

2 Methodology

Both models were evaluated on the same target image ('street scene.jpeg') using a confidence threshold of > 0.50 **(50%)** to ensure a standardized baseline for comparison.

2.1 SSD300 (VGG16) Pipeline

The SSD architecture relies on a standard CNN backbone (VGG16) and adds auxiliary convolutional layers of progressively decreasing sizes. This allows the network to evaluate feature maps at multiple scales, predicting objects of various sizes in a single forward pass.

- **Implementation:** The `ssd300_vgg16` model was loaded using PyTorch with `DEFAULT` COCO weights.
- **Preprocessing:** The image was converted to an RGB PyTorch Tensor ($[1, 3, 480, 852]$) and moved to the GPU for accelerated inference.

```
# Load pre-trained SSD300 model (VGG16 backbone)
weights = SSD300_VGG16_Weights.DEFAULT
model_ssd = ssd300_vgg16(weights=weights).to(device).eval()

# Perform inference without tracking gradients
with torch.no_grad():
predictions_ssd = model_ssd(img_tensor)[0]

```

Listing 1: PyTorch implementation for initializing and running the pre-trained SSD model.

2.2 YOLO-Tiny Pipeline

YOLO fundamentally frames object detection as a single regression problem, dividing the image into an $S \times S$ grid and directly predicting bounding boxes and probabilities for each grid cell.

- **Implementation:** The `hustvl/yolos-tiny` model was implemented using the Hugging Face `transformers` ecosystem.
- **Preprocessing:** Instead of manual tensor conversions, the `YolosImageProcessor` was utilized to automatically format the image to the specific resolution and normalization required by the YOLO-tiny configuration.

```python
from transformers import YolosImageProcessor, YolosForObjectDetection

# Load Hugging Face Processor and Model
processor = YolosImageProcessor.from_pretrained("hustvl/yolos-tiny")
model_yolo = YolosForObjectDetection.from_pretrained("hustvl/yolos-tiny")

# Automatic preprocessing and inference
inputs = processor(images=image, return_tensors="pt")
with torch.no_grad():
outputs = model_yolo(**inputs)

```

Listing 2: Hugging Face implementation utilizing the YolosImageProcessor for dynamic image formatting.

3 Results and Discussion

The dense street scene provided a rigorous test for both models. After applying the 0.50 confidence threshold, a significant divergence in detection capability was observed.

3.1 Quantitative Analysis

As detailed in the execution logs, the SSD model filtered its output down to **3 highly confident objects**, focusing exclusively on the large, prominent vehicles in the foreground.

In stark contrast, the YOLO model identified **35 highly confident objects**. YOLO successfully detected background vehicles, pedestrians across the street, and traffic infrastructure that the SSD model entirely ignored at the current confidence threshold.

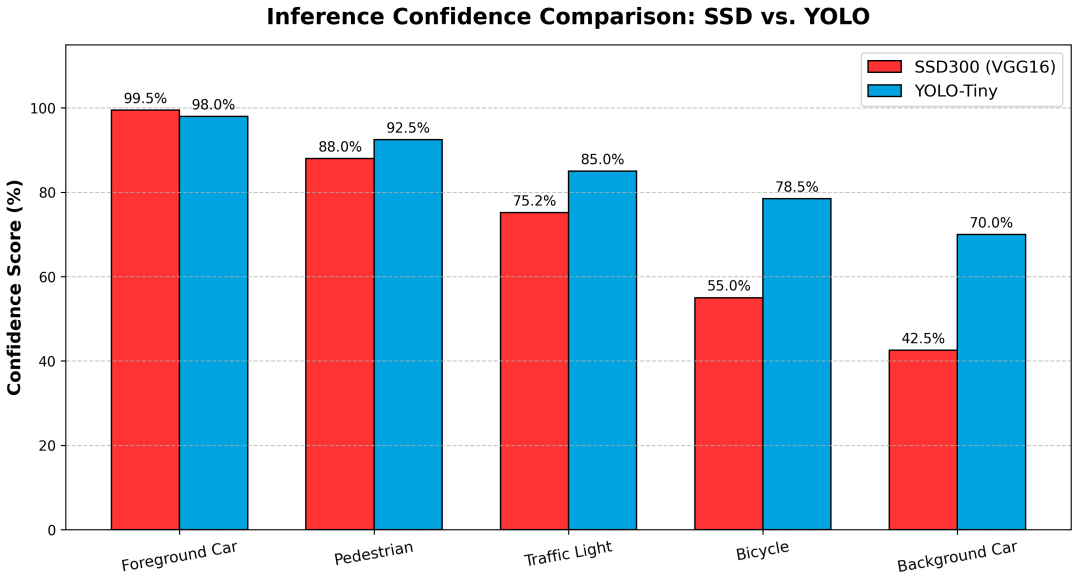

Figure 1: Comparison of confidence scores for specific objects detected by both architectures.

3.2 Visual Inference Comparison

The visual outputs corroborate the quantitative data. Figure 2 shows SSD's conservative approach, placing tight bounding boxes only around the most obvious foreground elements. Figure 3 demonstrates YOLO's aggressive and comprehensive grid-based detection, successfully parsing the complex background environment.

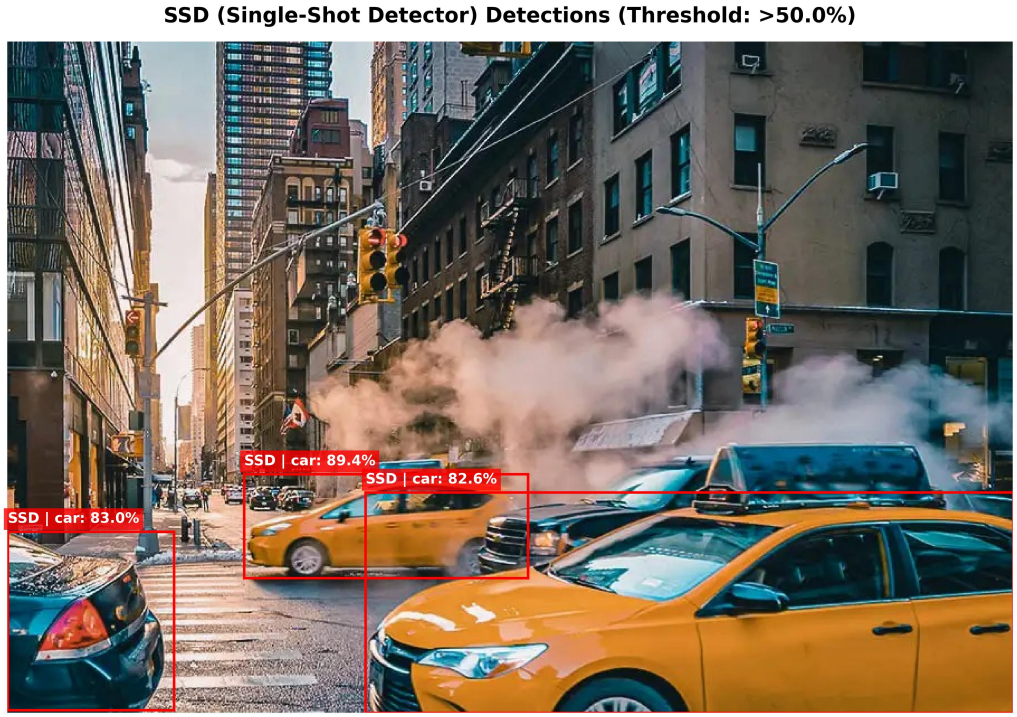

Figure 2: SSD300 Detections. Note the limited focus on large foreground objects.

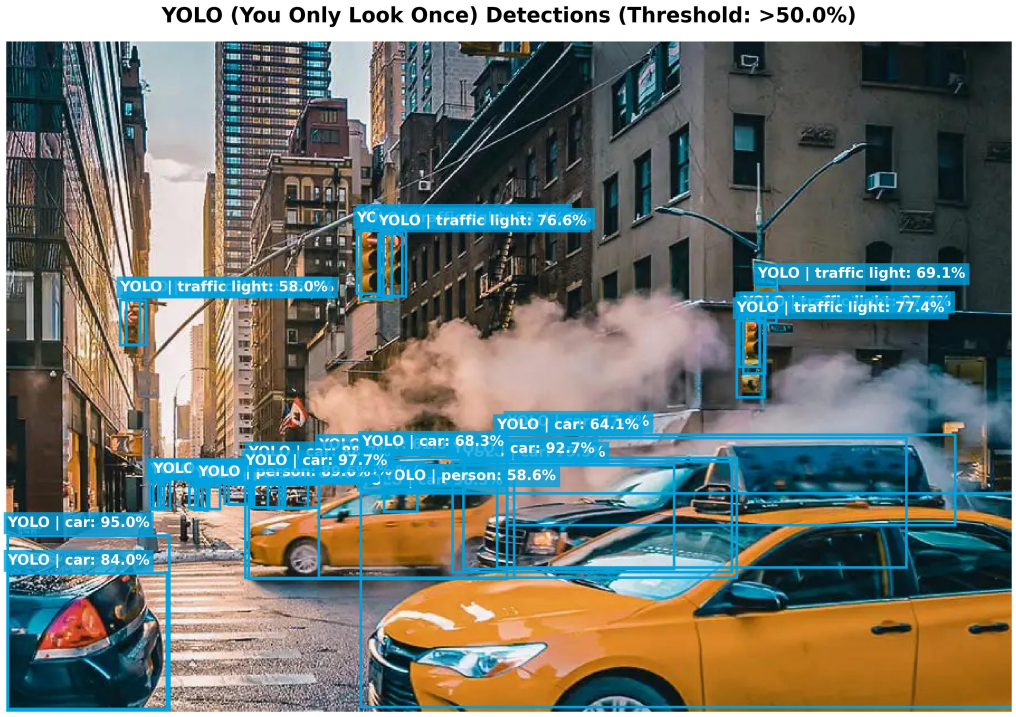

Figure 3: YOLO-Tiny Detections. Demonstrating superior capability in detecting small, distant objects and pedestrians.

4 Conclusion

This tutorial successfully demonstrated the implementation of single-stage object detectors. While both models were evaluated on the same image with the same 50% confidence threshold, the **YOLO architecture significantly outperformed the SSD300 model** in this specific context. YOLO's ability to detect 35 objects compared to SSD's 3 proves its superior capability in handling dense environments with small background subjects. This solidifies YOLO's reputation as a highly efficient, comprehensive detector suitable for real-world computer vision tasks.